Universidad de la Fuerzas Armadas - ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis Y Diseño de Software - NRC: 30723

Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML

Grupo: 4

Tufiño Erick

Santi Jeancarlo

Cedeño Andrés

Profesor: Ing. Jenny Ruiz

Fecha: 04 de Mayo de 2026

1. Casos de uso

1.1. Requisitos Funcionales de nivel 0

- RF-01 (Administrar Sistema): El sistema permite gestionar acceso, autenticación y configuración de perfiles.
- RF-02 (Gestionar Copropietarios): El sistema permite al Administrador importar, consultar, modificar y eliminar datos de copropietarios (CRUD de Copropietarios).
- RF-03 (Implementar Pagos): El sistema permite registrar comprobantes, validar pagos, calcular mora del 12% y actualizar estados de cuenta.
- RF-04 (Enviar Notificaciones): El sistema envía alertas automáticas por WhatsApp sobre la validación de pagos.

1.2. Especificación

Actores Principales: Administrador y Copropietario.

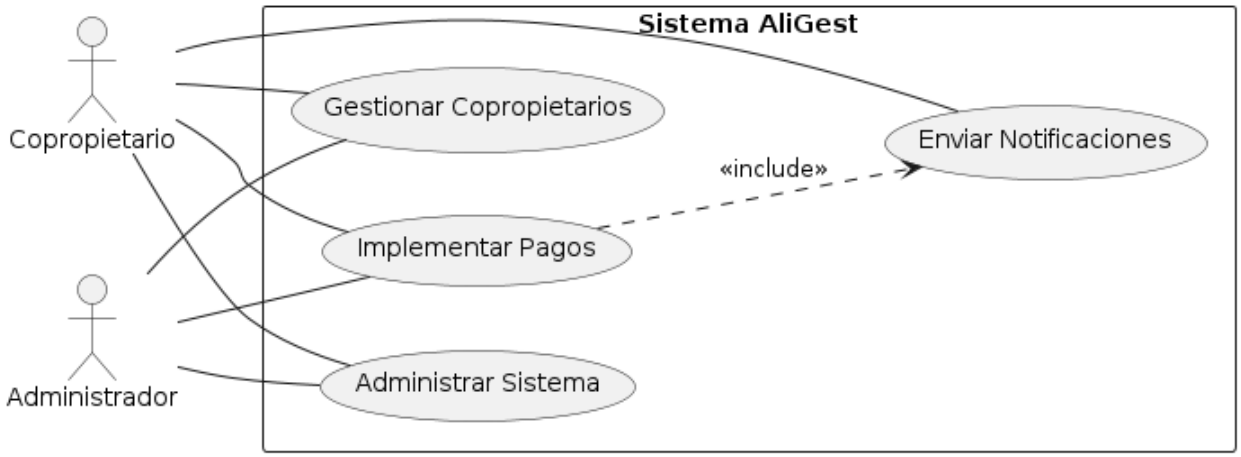
Administrar Sistema: Usuario ingresa credenciales -> el sistema valida, crea sesión y asigna rol.

Gestionar Copropietarios: Administrador sube Excel o edita datos -> el sistema valida (Casa única, cédula) -> guarda en BD. (Resultado: Copropietarios registrados con credenciales generadas).

Implementar Pagos (Realizar y Validar): Copropietario sube comprobante -> Administrador revisa -> el sistema verifica morosidad y aplica 12% -> Administrador aprueba. (Resultado: Pago registrado, deuda cubierta o remanente asignado, comprobante PDF generado).

Enviar Notificaciones: Se invoca automáticamente cuando un pago es validado (aprobado o rechazado), componiendo un mensaje y enviándolo por la API de WhatsApp.

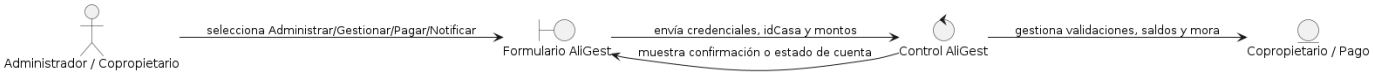
1.3. Diagrama de Casos de Uso



Código PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor Copropietario
actor Administrador

rectangle "Sistema AliGest" {
  usecase "Administrar Sistema" as UC1
  usecase "Gestionar Copropietarios" as UC2
  usecase "Implementar Pagos" as UC3
  usecase "Enviar Notificaciones" as UC4
}

Copropietario -- UC1
Copropietario -- UC2
Copropietario -- UC3

Administrador -- UC1
Administrador -- UC2
Administrador -- UC3

UC4 -- Copropietario
UC3 ..> UC4 : <<include>>
@enduml
```

1.4. Puente



```
@startuml
left to right direction

actor "Administrador / Copropietario" as Actor
boundary "Formulario AliGest" as UI
control "Control AliGest" as Ctrl
entity "Copropietario / Pago" as Ent

Actor --> UI : selecciona Administrar/Gestionar/Pagar/Notificar
UI --> Ctrl : envía credenciales, idCasa y montos
Ctrl --> Ent : gestiona validaciones, saldos y mora
Ctrl --> UI : muestra confirmación o estado de cuenta
@enduml
```

2. Clases de análisis

2.1. Responsabilidades Identificadas

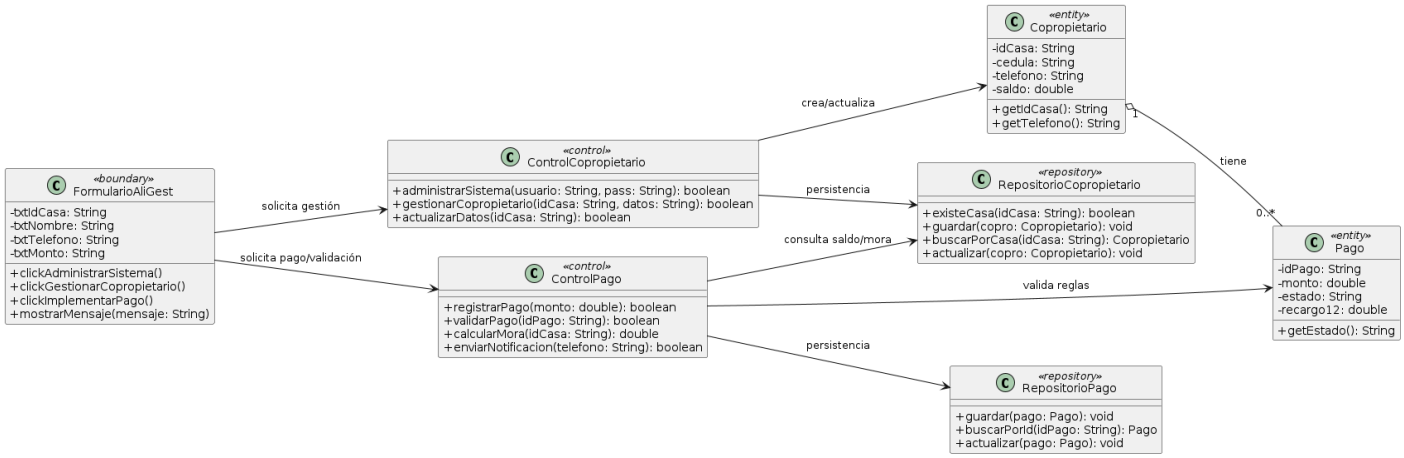
Frontera (Boundary) - PantallaGestionPagosCopropietarios: Representa las interfaces de usuario (web) donde el Administrador aprueba pagos y gestiona residentes, y el Copropietario sube sus transferencias.

Control - ControladorFinancieroAliGest: Concentra la lógica de validación de archivos, el cálculo automático de la mora (12%), el procesamiento del comprobante y la orquestación del envío de WhatsApp.

Entidad (Entity) - Copropietario y PagoExpensa: Representan la información central del condominio (Casa, cédula, estado de cuenta, monto base, recargo, fecha).

Repositorio - RepositorioAliGest: Gestiona el acceso y almacenamiento persistente en la Base de Datos para copropietarios y transacciones financieras.

2.2. Diagrama de Clases de Análisis



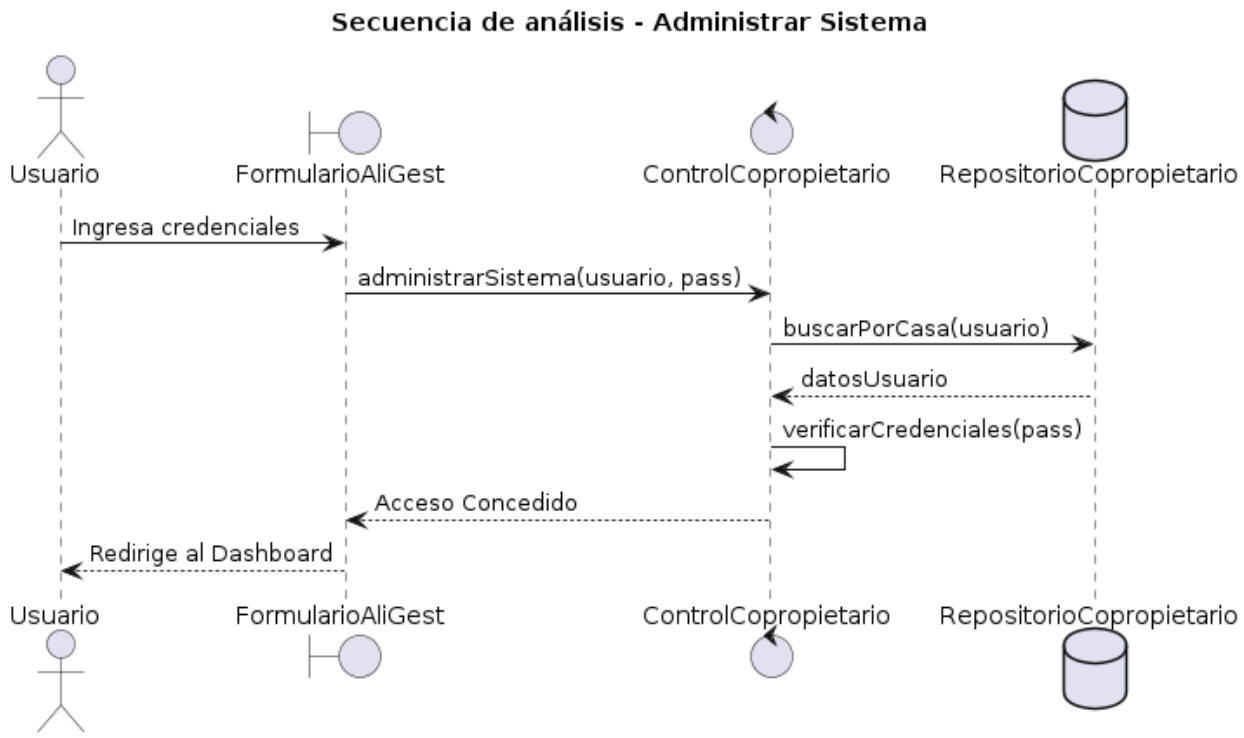
Código PlantUML

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioAliGest" <<boundary>> {
  - txtIdCasa: String
  - txtNombre: String
  - txtTelefono: String
  - txtMonto: String
  + clickAdministrarSistema()
  + clickGestionarCopropietario()
  + clickImplementarPago()
  + mostrarMensaje(mensaje: String)
}

class "ControlCopropietario" <<control>> {
  + administrarSistema(usuario: String, pass: String): boolean
  + gestionarCopropietario(idCasa: String, datos: String): boolean
  + actualizarDatos(idCasa: String): boolean
}

class "ControlPago" <<control>> {
  + registrarPago(monto: double): boolean
  + validarPago(idPago: String): boolean
  + calcularMora(idCasa: String): double
  + enviarNotificacion(telefono: String): boolean
}

class "Copropietario" <<entity>> {
  - idCasa: String
  - cedula: String
  - telefono: String
  - saldo: double
```

```
  + getIdCasa(): String
  + getTelefono(): String
}

class "Pago" <<entity>> {
  - idPago: String
  - monto: double
  - estado: String
  - recargo12: double
  + getEstado(): String
}

class "RepositorioCopropietario" <<repository>> {
  + existeCasa(idCasa: String): boolean
  + guardar(copro: Copropietario): void
  + buscarPorCasa(idCasa: String): Copropietario
  + actualizar(copro: Copropietario): void
}

class "RepositorioPago" <<repository>> {
  + guardar(pago: Pago): void
  + buscarPorId(idPago: String): Pago
  + actualizar(pago: Pago): void
}

FormularioAliGest --> ControlCopropietario : solicita gestión
FormularioAliGest --> ControlPago : solicita pago/validación

ControlCopropietario --> Copropietario : crea/actualiza
ControlCopropietario --> RepositorioCopropietario : persistencia

ControlPago --> Pago : valida reglas
ControlPago --> RepositorioPago : persistencia
ControlPago --> RepositorioCopropietario : consulta saldo/mora

Copropietario "1" o-- "0..*" Pago : tiene
@enduml
```

3. Secuencias de análisis

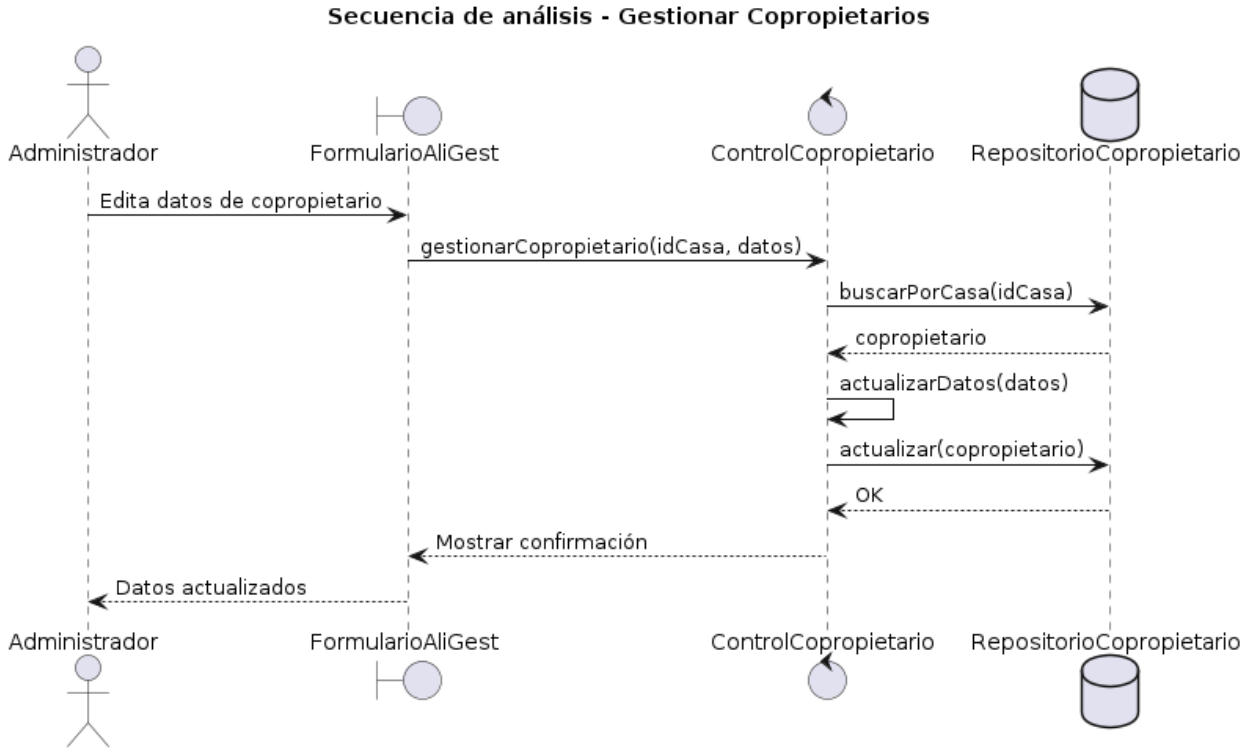
3.1. Secuencia: Administrar Sistema (Inicio de Sesión)



Código PlantUML

```
@startuml
title Secuencia de análisis - Administrar Sistema
actor "Usuario" as Actor
boundary "FormularioAliGest" as UI
control "ControlCopropietario" as Ctrl
database "RepositorioCopropietario" as Repo

Actor -> UI : Ingresa credenciales
UI -> Ctrl : administrarSistema(usuario, pass)
Ctrl -> Repo : buscarPorCasa(usuario)
Repo --> Ctrl : datosUsuario
Ctrl -> Ctrl : verificarCredenciales(pass)
Ctrl --> UI : Acceso Concedido
UI --> Actor : Redirige al Dashboard
@enduml
```

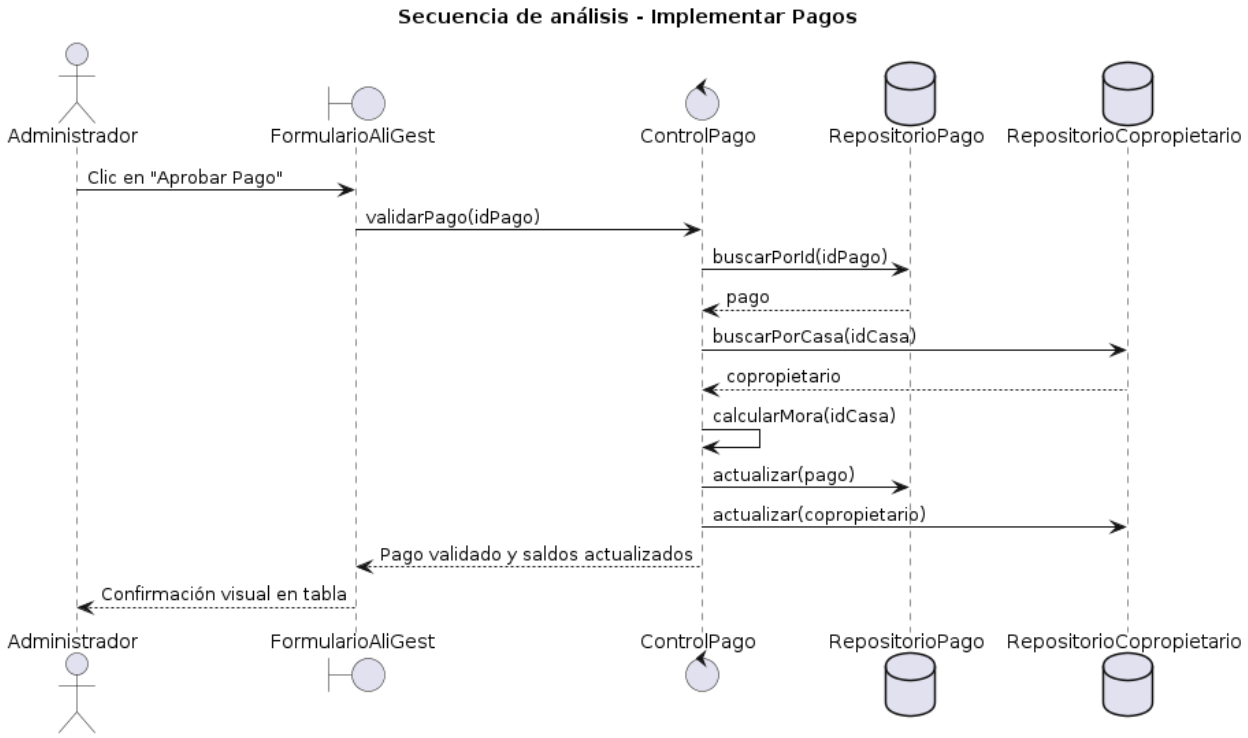
3.2. Secuencia: Gestionar Copropietarios (Modificar Registro)



Código PlantUML

```
@startuml
title Secuencia de análisis - Gestionar Copropietarios
actor "Administrador" as Admin
boundary "FormularioAliGest" as UI
control "ControlCopropietario" as Ctrl
database "RepositorioCopropietario" as Repo

Admin -> UI : Edita datos de copropietario
UI -> Ctrl : gestionarCopropietario(idCasa, datos)
Ctrl -> Repo : buscarPorCasa(idCasa)
Repo --> Ctrl : copropietario
Ctrl -> Ctrl : actualizarDatos(datos)
Ctrl -> Repo : actualizar(copropietario)
Repo --> Ctrl : OK
Ctrl --> UI : Mostrar confirmación
UI --> Admin : Datos actualizados
@enduml
```

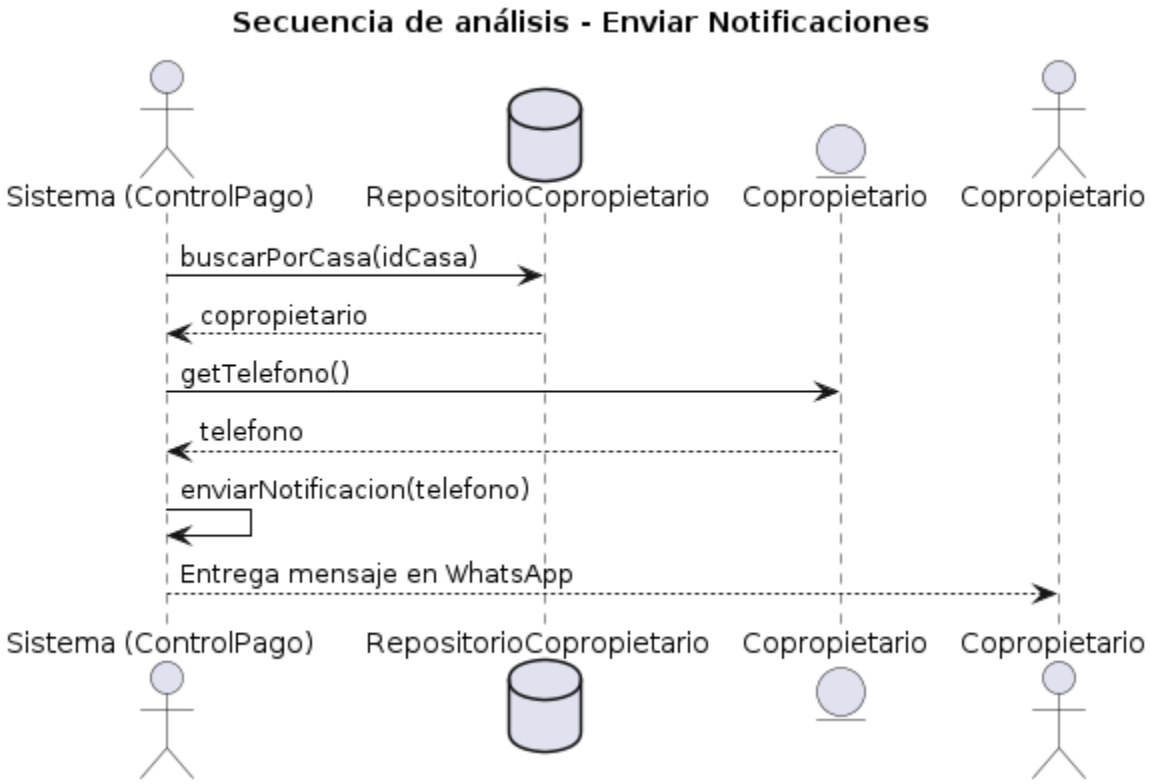
3.3. Secuencia: Implementar Pagos (Realizar y Validar)



Código PlantUML

```
@startuml
title Secuencia de análisis - Implementar Pagos
actor "Administrador" as Admin
boundary "FormularioAliGest" as UI
control "ControlPago" as Ctrl
database "RepositorioPago" as RepoPago
database "RepositorioCopropietario" as RepoCopro

Admin -> UI : Clic en "Aprobar Pago"
UI -> Ctrl : validarPago(idPago)
Ctrl -> RepoPago : buscarPorId(idPago)
RepoPago --> Ctrl : pago
Ctrl -> RepoCopro : buscarPorCasa(idCasa)
RepoCopro --> Ctrl : copropietario
Ctrl -> Ctrl : calcularMora(idCasa)
Ctrl -> RepoPago : actualizar(pago)
Ctrl -> RepoCopro : actualizar(copropietario)
Ctrl --> UI : Pago validado y saldos actualizados
UI --> Admin : Confirmación visual en tabla
@enduml
```

3.4. Secuencia: Enviar Notificaciones (Automático por WhatsApp)



Código PlantUML

```
@startuml
title Secuencia de análisis - Enviar Notificaciones
actor "Sistema (ControlPago)" as Trigger
database "RepositorioCopropietario" as Repo
entity "Copropietario" as Copro
actor "Copropietario" as Destinatario

Trigger -> Repo : buscarPorCasa(idCasa)
Repo --> Trigger : copropietario
Trigger -> Copro : getTelefono()
Copro --> Trigger : telefono
Trigger -> Trigger : enviarNotificacion(telefono)
Trigger --> Destinatario : Entrega mensaje en WhatsApp
@enduml
```

4. Trazabilidad

Matriz de relación integral:

RF (Nivel 0)	Caso de Uso	Clases de Análisis Participantes	Diagrama de Secuencia
RF-01	Administrar Sistema	PantallaAutenticacion, ControladorSistema, Usuario, RepositorioUsuarios	Secuencia - Administrar Sistema (Iniciar Sesión)
RF-02	Gestionar Copropietarios	PantallaCopropietarios, ControladorCopropietarios, Copropietario, RepositorioCopropietarios	Secuencia - Gestionar Copropietarios (Modificar Datos)
RF-03	Implementar Pagos	PantallaPagos, ControladorPagos, PagoExpensa, RepositorioPagos, RepositorioCopropietarios	Secuencia - Implementar Pagos (Validación y Mora)
RF-04	Enviar Notificaciones	ControladorPagos, ServicioWhatsApp, RepositorioCopropietarios	Secuencia - Enviar Notificaciones (Vía WhatsApp API)

5. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia demuestra que el modelado de software mantiene coherencia cuando cada elemento cumple su responsabilidad: los casos de uso definen qué requiere el actor (ej. registrar copropietarios o validar expensas), las clases establecen quién lo resuelve (ej. los Controladores ejecutando las reglas del negocio) y las secuencias muestran cómo colaboran los objetos en el tiempo. Esta coherencia se confirmó al mapear el SRS del proyecto AliGest, donde los 4 requerimientos principales de nivel 0 cuentan con su respectiva orquestación estructural y dinámica. Cada funcionalidad clave (desde la validación de la mora del 12% hasta el envío automatizado de mensajes de WhatsApp) está respaldada por secuencias exactas, lo que evidencia que un modelo de análisis bien construido no solo clarifica el diseño del sistema, sino que actúa como una guía directa y libre de ambigüedades para la fase de implementación bajo la metodología XP.